

APEX RETAIL INTELLIGENCE

Final Project Report

Celebal Technologies – Excellence Internship Program 2026

Enterprise Retail Data Engineering & Business Intelligence Platform

Project Area	Technology Focus
Data Engineering	Databricks, Apache Spark, Delta Lake
Architecture	Medallion Architecture: Raw / Bronze / Silver / Gold
Data Modeling	Dimensional Modeling / Star Schema
Data Quality	Audit Validation, PASS/FAIL Controls
Business Intelligence	Retail KPIs and analytical datasets
Change Management	MERGE-based incremental processing, SCD

Prepared as the final project deliverable for the Celebal Excellence Internship.

Project title: Apex Retail Intelligence

Project by: Samradni Ashok Dahiphale [CT_CSI_DE_1173]

GitHub:https://github.com/COffeebean2610/Data_Engineering_Internship_Celebal_Technologies

Internship Project Declaration

This report documents the final project developed as part of the Celebal Technologies Excellence Internship Program. The project, titled “Apex Retail Intelligence,” demonstrates the application of data engineering concepts learned during the internship, with emphasis on Databricks, Apache Spark, Delta Lake, Medallion Architecture, incremental processing, dimensional modeling, data quality, and business KPI generation.

The project is presented as an academic and internship learning deliverable. The implementation focuses on building a reliable analytical data platform from retail source files and converting raw transactional information into trusted, business-ready datasets.

Declaration: The report has been prepared to describe the architecture, implementation approach, transformations, validations, and analytical outcomes of the project. Any implementation-specific values, screenshots, or environment identifiers should be updated with the exact values from the final Databricks workspace before external submission.

Acknowledgement

I would like to express my sincere gratitude to Celebal Technologies for providing the Excellence Internship opportunity and for creating an environment in which practical data engineering skills could be developed through structured learning and hands-on implementation.

I am thankful to the internship mentors, trainers, and coordinators for their guidance across Python, SQL, data engineering, Azure concepts, ETL/ELT, Spark, Databricks, Delta Lake, data modeling, and pipeline design. The progression from foundational concepts to an integrated final project helped connect individual technical topics into an end-to-end engineering workflow.

I also acknowledge the support of my faculty, peers, and everyone who contributed feedback during the development and documentation of this project. Their suggestions helped improve the organization of the solution, clarity of the business KPIs, and completeness of the final validation process.

Finally, I am grateful for the opportunity to work on a project that combines engineering reliability with business intelligence. Apex Retail Intelligence represents the practical application of the concepts learned during the internship.

Abstract

Apex Retail Intelligence is an enterprise-style retail analytics platform designed to transform raw retail data into reliable, structured, and business-ready information. The solution addresses a common data engineering challenge: retail organizations generate data from transactions, customers, products, stores, regions, and operational events, but raw files are not immediately suitable for analytics.

The project implements a layered Medallion Architecture using Databricks and Delta Lake. Source files first enter a controlled raw/landing area. The Bronze layer preserves ingested information in a Delta-based structure. The Silver layer applies cleansing, standardization, duplicate handling, datatype correction, business rules, and incremental upsert logic. The Gold layer organizes curated information using a dimensional model and exposes analytical datasets for business intelligence.

The project also introduces audit validation controls that classify data-quality checks as PASS or FAIL and can stop downstream processing when critical validation conditions are not satisfied. Customer history is managed using Slowly Changing Dimension Type 2 logic, while product attributes are handled using Type 1 logic where appropriate.

The final analytical layer supports retail KPIs including Regional Net Margin, Promotional Effectiveness through Average Order Value, Customer Churn, and Store Traffic. Together, these outputs provide a foundation for management reporting, operational analysis, and future dashboard development.

Table of Contents

1. Introduction
2. Project Objectives and Scope
3. Business Problem
4. Requirement Analysis
5. Technology Stack
6. Solution Architecture
7. Data Sources and Ingestion
8. Raw and Bronze Layer
9. Silver Transformation Layer
10. Gold Data Warehouse Layer
11. Dimensional Modeling
12. Slowly Changing Dimensions
13. Incremental Processing and MERGE
14. Data Quality and Audit Framework
15. Retail KPI Design
16. KPI 1 – Regional Net Margin
17. KPI 2 – Promotional Effectiveness
18. KPI 3 – Customer Churn
19. KPI 4 – Store Traffic
20. Notebook and Pipeline Design
21. Security, Governance and Reliability
22. Testing Strategy
23. Validation Results
24. Performance Considerations
25. Challenges and Solutions
26. Business Value
27. Limitations and Future Scope
28. Conclusion
29. Learning Outcomes
30. References and Appendix

1. Introduction

Retail data engineering sits at the intersection of operational systems and business decision-making. Transaction records may be generated continuously, while customer, product, store, and promotion information changes over time. A useful analytics solution must therefore do more than simply load files into a database. It must preserve source data, validate incoming records, apply repeatable transformations, manage changes, and publish trusted datasets.

Apex Retail Intelligence was conceived as an end-to-end data engineering project to demonstrate these principles in a practical retail context. The platform takes source data through multiple controlled layers, ensuring that each stage has a defined purpose and quality expectation.

1.1 Motivation

The motivation for the project is to create a reusable analytical foundation for retail organizations. Instead of allowing every report or analyst to independently clean source files, the platform centralizes data preparation and business logic. This improves consistency and reduces the risk of different teams calculating the same KPI differently.

1.2 Project Perspective

The project follows an engineering-first approach. Data is treated as a lifecycle: ingestion, storage, validation, transformation, modeling, analytics, and monitoring. The architecture is intentionally modular so that additional sources, dimensions, KPIs, or downstream reporting tools can be added without redesigning the complete pipeline.

1.3 Expected Outcome

- Reliable and traceable ingestion of retail source files.
- Cleaned and standardized datasets suitable for analytical use.
- Incremental processing without repeatedly rebuilding complete tables.
- Historical tracking for selected changing dimensions.
- Business-ready star schema structures.
- Validated KPI datasets for retail decision support.

2. Project Objectives and Scope

2.1 Primary Objectives

1. Build a complete retail data engineering workflow in Databricks.
2. Implement a Medallion Architecture using Raw, Bronze, Silver, and Gold stages.
3. Use Delta Lake capabilities for reliable storage and transactional updates.
4. Apply cleansing, standardization, deduplication, and business transformations.
5. Implement incremental loading and MERGE-based upsert processing.
6. Apply SCD Type 2 logic to preserve customer history and Type 1 handling for product changes.
7. Create a star-schema-oriented Gold layer for analytics.
8. Develop and validate business KPIs relevant to retail operations.
9. Introduce audit checks that can prevent invalid data from reaching downstream layers.

2.2 Scope

The scope includes source-file ingestion, data validation, transformation, dimensional modeling, KPI calculation, and final testing. The project is primarily focused on the data platform rather than a full production BI dashboard or enterprise orchestration service. The Gold datasets are designed to be consumed by a BI layer in a future extension.

2.3 Out of Scope

- Real-time streaming implementation.
- Production-scale identity and access management configuration.
- Enterprise master-data management.
- External customer-facing application development.
- Automated machine-learning forecasting in the core pipeline.

3. Business Problem

Retail businesses need to understand profitability, promotions, customer retention, and store performance. These questions sound simple but require data from multiple business entities. Revenue must be connected with cost to estimate margin. Orders must be associated with customers to understand retention. Promotions must be linked to transactions to evaluate effectiveness. Store activity must be aggregated to assess traffic and operational performance.

When source data exists as independent CSV files or operational extracts, common problems include inconsistent column names, missing values, duplicate records, incompatible data types, late-arriving records, changed customer attributes, and repeated calculations of business metrics.

3.1 Key Business Questions

- Which regions contribute the strongest net margin?
- Do promoted transactions generate higher average order value?
- Which customers are at risk of churn based on activity patterns?
- How does store traffic vary across stores and time periods?
- Can management rely on the same KPI definition across reports?

3.2 Engineering Response

The solution addresses these questions by creating a trusted analytical layer. Rather than calculating every metric directly from raw data, the project establishes governed transformations and reusable Gold datasets. This separates data engineering responsibilities from reporting and allows business users to consume standardized outputs.

4. Requirement Analysis

Requirement	Description	Implementation Approach
Scalable ingestion	Load retail source files reliably	Databricks ingestion workflow
Data preservation	Retain source-level information	Raw/Bronze Delta storage
Data quality	Detect invalid records early	Audit and validation checks
Cleansing	Standardize and correct records	Silver transformations
Incremental processing	Process only new/changed data	MERGE-based upserts
Historical tracking	Preserve selected dimension history	SCD Type 2
Analytics model	Provide business-friendly structure	Gold star schema
KPI generation	Calculate defined retail measures	Gold analytical queries
Validation	Verify outputs before delivery	Final testing notebook

4.1 Functional Requirements

- Load historical and incremental retail source files.
- Maintain unique business keys where required.
- Standardize dates, numeric fields, and categorical attributes.
- Handle duplicates and null values according to business rules.
- Merge new and changed records into curated tables.
- Produce dimension and fact structures.
- Calculate and validate four major retail KPIs.

4.2 Non-Functional Requirements

- Repeatability: the pipeline should be rerunnable.
- Traceability: source-to-target transformations should be understandable.
- Reliability: failed critical checks should stop downstream processing.
- Maintainability: logic should be separated by pipeline stage.
- Extensibility: additional KPIs and dimensions should be possible without major redesign.

5. Technology Stack

Technology	Role in Project
Databricks	Primary development and execution environment
Apache Spark	Distributed processing and transformations
Delta Lake	Transactional storage, MERGE, versioning and reliable tables
PySpark / SQL	Data transformation and analytical logic
Python	Notebook logic, validation and orchestration support
Unity Catalog	Data object governance and registration
CSV	Source data exchange format
Git / GitHub	Version control and project documentation
BI Layer (future-ready)	Consumption of Gold analytical datasets

5.1 Why Databricks?

Databricks provides an integrated environment for Spark-based data processing, SQL analytics, notebooks, Delta Lake, and governance. For this project it reduces the gap between engineering and analytics by allowing the same platform to support ingestion, transformation, validation, and analytical querying.

5.2 Why Delta Lake?

Delta Lake adds reliability to data-lake storage through transactional semantics, schema enforcement, versioned data, and efficient update patterns. These capabilities are particularly useful for incremental retail processing, where records may need to be inserted or updated rather than appended blindly.

6. Solution Architecture

The architecture follows a Medallion pattern. Each layer has a defined responsibility and quality expectation.

Layer	Purpose	Typical Content
Raw / Landing	Initial receipt and source preservation	Original CSV extracts
Bronze	Structured ingestion with minimal alteration	Source-aligned Delta tables
Silver	Cleansing, standardization and business rules	Validated transactional and master data
Gold	Curated analytical model	Facts, dimensions and KPI datasets

6.1 End-to-End Flow

Source CSV → Raw/Landing → Bronze → Audit Validation → Silver Transformation → MERGE/SCD Processing → Gold Star Schema → KPI Datasets → BI/Reporting

6.2 Architectural Principles

- Preserve before transforming.
- Validate before publishing.
- Separate engineering layers from analytical outputs.
- Use stable business keys for incremental processing.
- Make business logic explicit and reusable.
- Fail safely when critical data-quality rules are violated.

6.3 Architecture Diagram – Suggested Figure

Insert the final Architecture.png from the project repository here. The figure should show source files flowing through Raw/Landing, Bronze, Silver, and Gold, followed by KPI and BI consumption.

7. Data Sources and Ingestion

The project uses retail source files representing operational entities required for analysis. The source format is CSV, which is common for extracts exchanged between systems or provided for batch analytics exercises.

7.1 Representative Source Entities

Entity	Purpose	Examples of Attributes
Customers	Customer master and history	Customer ID, name, segment, location
Products	Product master	Product ID, category, price, cost
Stores	Store master	Store ID, region, location
Transactions / Orders	Sales activity	Order ID, customer, store, date, amount
Promotions	Promotion information	Promotion ID, type, discount
Traffic	Store activity	Store, date, visitors / traffic count

7.2 Historical vs Incremental Files

Historical files establish the initial state of the platform. Incremental files represent later arrivals and are processed using controlled update logic. Separating these modes is important because a production-style pipeline should not require complete reprocessing whenever a small number of records arrive.

7.3 Ingestion Principles

- Validate file availability before processing.
- Capture source-aligned data in Bronze.
- Preserve identifiers required for downstream joins.
- Record ingestion metadata where applicable.
- Do not apply complex business logic in the raw ingestion stage.

8. Raw and Bronze Layer

The Raw/Landing area is the first controlled boundary between external files and the analytical platform. The objective is preservation rather than aggressive transformation. The Bronze layer then stores the ingested records in a structured Delta representation.

8.1 Bronze Design Goals

- Maintain a faithful representation of source records.
- Support repeatable downstream processing.
- Enable auditability and traceability.
- Provide a stable input to Silver transformations.
- Use Delta format to support reliable table operations.

8.2 Bronze Transformations

Bronze transformations are intentionally limited. Typical operations include reading CSV data with an explicit schema or controlled inference, standardizing technical metadata, and writing the records to Delta tables. Business calculations are deferred to Silver and Gold.

8.3 Why Not Clean Everything in Bronze?

Keeping Bronze close to the source preserves an important audit boundary. If a business rule changes later, the original ingested representation remains available for reprocessing. This reduces the risk of losing information through irreversible early transformations.

9. Silver Transformation Layer

Silver is the principal data-quality and transformation layer. Here, raw records are converted into consistent, analytics-ready entities. The transformation stage applies business and technical rules while retaining enough structure to trace results back to Bronze.

9.1 Common Silver Operations

- Null handling according to field-level business requirements.
- Duplicate identification and removal.
- Datatype correction for dates, amounts and identifiers.
- Column standardization.
- Whitespace and text normalization.
- Derived field creation.
- Validation of required keys.
- Filtering of invalid records.
- Joining related datasets when required for enrichment.

9.2 Example Transformation Pattern

A transaction amount may arrive as text. Silver processing converts it to a numeric type, validates non-negative expectations where applicable, and creates a standardized monetary field. Date strings are converted to a consistent date/timestamp representation so that downstream aggregations do not depend on source formatting.

9.3 Silver Quality Gate

Before Gold processing, the project applies audit checks. These checks are designed to identify issues that could materially affect business outputs. A critical validation failure should stop or quarantine downstream processing rather than silently publish questionable results.

10. Gold Data Warehouse Layer

The Gold layer is designed for consumption by analysts, reporting tools, and KPI queries. It contains curated entities and analytical relationships that are easier for business users to understand than source-oriented tables.

10.1 Gold Objectives

- Business-friendly naming and structure.
- Stable relationships between facts and dimensions.
- Reusable measures and attributes.
- Efficient aggregation for BI use cases.
- Clear ownership of KPI definitions.

10.2 Gold Outputs

The project uses a dimensional approach with fact and dimension structures. Typical dimensions include Customer, Product, Store, Date, Promotion, and Region, while the principal fact structures represent retail transactions and operational activity.

10.3 BI Readiness

Gold datasets are designed so a BI developer can connect a reporting layer without rebuilding the core transformation logic. The same KPI definitions can therefore be reused across dashboards, management reports, and ad-hoc analysis.

11. Dimensional Modeling

Dimensional modeling organizes analytical data around measurable business events and descriptive dimensions. A star schema is used because it provides a simple structure for analytical queries and makes relationships intuitive for BI users.

11.1 Fact Tables

Fact tables contain measurable events such as sales transactions, order amounts, discounts, quantities, and traffic counts. They typically reference dimensions through keys and store numeric measures used in aggregation.

11.2 Dimension Tables

Dimension tables provide descriptive context such as customer attributes, product categories, store regions, dates, and promotion characteristics.

Table Type	Example	Purpose
Fact	Fact Sales	Measures sales, quantity, discount, cost
Fact	Fact Traffic	Measures store visits/activity
Dimension	Dim Customer	Customer attributes and history
Dimension	Dim Product	Product descriptive attributes
Dimension	Dim Store	Store and regional context
Dimension	Dim Date	Calendar attributes
Dimension	Dim Promotion	Promotion context

11.3 Benefits

- Simple joins for BI users.
- Clear separation between measures and descriptive attributes.
- Efficient aggregation patterns.
- Reusable dimensions across multiple facts.

12. Slowly Changing Dimensions

Dimension attributes can change over time. For example, a customer's segment, location, or other descriptive attributes may change. If the business needs to preserve historical context, simply overwriting the old value is insufficient.

12.1 Customer – SCD Type 2

The project applies Type 2 logic to customer history. When a tracked attribute changes, the existing customer version is closed and a new version is inserted. A typical implementation uses a surrogate key, effective start date, effective end date, and current-record indicator.

Column	Purpose
Customer Surrogate Key	Unique identifier for a version of the customer
Customer Business Key	Stable source/customer identifier
Effective Start	Beginning of record validity
Effective End	End of record validity
Current Flag	Identifies the active version

12.2 Product – SCD Type 1

Product attributes that do not require historical preservation can use Type 1 handling. When a value changes, the current record is updated. This is appropriate when the business is interested in the latest product description rather than historical versions.

12.3 Business Benefit

Using different SCD strategies demonstrates that historical requirements should be driven by business meaning rather than applying one method universally.

13. Incremental Processing and MERGE

Incremental processing is central to a practical data platform. Instead of rebuilding an entire table for every delivery, new and changed records are identified and applied to the target.

13.1 MERGE Concept

A MERGE operation compares incoming records with target records using a business key. When a match exists, the record can be updated according to the business rule. When no match exists, the record can be inserted.

13.2 Simplified Logic

Condition	Action
Source key matches target and record changed	Update target / close old version when SCD2 applies
Source key does not match target	Insert new record
Source record is unchanged	No unnecessary update
Critical source validation fails	Stop or quarantine downstream processing

13.3 Advantages

- Reduced processing compared with full rebuilds.
- Supports repeatable batch execution.
- Handles inserts and updates in one logical operation.
- Works naturally with Delta Lake transactional tables.
- Supports SCD implementation patterns.

14. Data Quality and Audit Framework

A data pipeline is only as reliable as the data it publishes. Apex Retail Intelligence includes an audit validation stage to detect issues before Gold outputs are generated.

14.1 Validation Categories

Check	Example Rule	Expected Result
Row count	Source and target counts within expected bounds	PASS
Null check	Required keys are not null	PASS
Duplicate check	Business key uniqueness	PASS
Datatype check	Numeric/date fields match expected type	PASS
Referential integrity	Fact keys resolve to dimensions	PASS
Business rule	Amounts and quantities satisfy defined rules	PASS

14.2 PASS/FAIL Pattern

Each audit condition can be represented as a PASS or FAIL result. Critical failures should be treated as pipeline-control events. This prevents a technically successful job from being interpreted as a successful business pipeline when the output is actually unreliable.

14.3 Audit Benefits

- Early detection of data issues.
- Clear evidence for final validation.
- Improved trust in KPI outputs.
- Repeatable quality gates.
- Foundation for future automated monitoring.

15. Retail KPI Design

The Gold layer supports four core retail KPIs selected to demonstrate the transition from data engineering to business intelligence. Each KPI requires well-defined inputs and consistent business logic.

KPI	Business Question	Primary Inputs
Regional Net Margin	Which regions are most profitable?	Revenue, cost, region
Promotional Effectiveness	Do promoted orders have stronger AOV?	Order value, promotion flag
Customer Churn	Which customers have become inactive?	Customer, order dates
Store Traffic	How much activity does each store receive?	Store, date, traffic

15.1 KPI Governance

A KPI is not simply a SQL expression. Its definition, filters, grain, and treatment of missing values should be documented. This project therefore treats KPI logic as part of the Gold data product rather than as an informal report calculation.

16. KPI 1 – Regional Net Margin

Regional Net Margin measures profitability after accounting for relevant costs. It provides management with a regional view of financial performance.

16.1 Conceptual Formula

Net Margin = Revenue – Cost

Net Margin % = (Revenue – Cost) / Revenue × 100

16.2 Processing Steps

10. Join transaction facts with product cost information.
11. Associate each transaction with the relevant store and region.
12. Aggregate revenue and cost by region.
13. Calculate net margin and, where required, margin percentage.
14. Validate that denominator and required fields are present.

16.3 Interpretation

A region with high revenue is not automatically the most profitable. Comparing revenue with cost reveals whether sales volume is translating into financial contribution. The KPI can therefore support decisions related to regional strategy, product mix, and store performance.

17. KPI 2 – Promotional Effectiveness

Promotional Effectiveness is represented through Average Order Value (AOV) comparisons between promoted and non-promoted orders.

17.1 Formula

$$\text{AOV} = \text{Total Order Value} / \text{Number of Orders}$$

17.2 Analysis

Orders are grouped by promotion status. The Gold dataset can expose promoted AOV, non-promoted AOV, order counts, and the difference between the two groups.

17.3 Business Interpretation

- A higher promoted AOV may indicate successful basket expansion.
- A lower promoted AOV may indicate discount-driven purchasing without sufficient basket growth.
- Results should be interpreted together with margin because higher AOV does not necessarily mean higher profitability.
- Promotion analysis can be extended to promotion type, product category, store, and region.

17.4 Example Analytical Output

Segment	Orders	AOV	Interpretation
Promoted	Calculated from Gold facts	Calculated	Evaluate basket impact
Non-Promoted	Calculated from Gold facts	Calculated	Baseline comparison
Difference	Derived	Promoted AOV – Baseline	Measure promotional uplift

18. KPI 3 – Customer Churn

Customer churn identifies customers who have become inactive according to a defined inactivity window. Because churn depends on time, the definition should be documented explicitly.

18.1 Conceptual Approach

15. Determine each customer's most recent transaction date.
16. Compare the latest activity date with the analysis/reference date.
17. Apply the selected inactivity threshold.
18. Classify customers as active or churned.
19. Aggregate churn counts and rates by relevant customer attributes.

18.2 Important Considerations

Churn is a business definition, not a universal mathematical constant. A retail business with weekly purchases may define inactivity differently from a business with annual purchases. The threshold should therefore be parameterized in a production implementation.

18.3 Business Value

- Prioritize retention campaigns.
- Identify customer segments with elevated churn.
- Measure retention trends over time.
- Connect customer churn with promotions and product categories.

19. KPI 4 – Store Traffic

Store Traffic measures the volume of store activity over a selected period. It can be used alongside sales to examine conversion and operational performance.

19.1 Calculation

Store Traffic = Sum of valid traffic/visitor observations for the selected store and period.

19.2 Analytical Dimensions

- Store
- Region
- Date
- Week / month
- Promotion period
- Sales performance

19.3 Extended Insight

Traffic becomes more valuable when compared with sales. A store with high traffic but relatively low sales may indicate a conversion opportunity, while a store with lower traffic and strong sales may have a different customer mix or purchasing behavior.

20. Notebook and Pipeline Design

The project is organized into purpose-specific notebooks to improve maintainability and demonstrate separation of concerns.

Notebook	Responsibility
01_Raw_Landing_Audit	Source ingestion, landing controls and initial validation
02_Silver_Transformation_MERGE	Cleansing, standardization, incremental processing and MERGE
03_Gold_StarSchema_KPIs	Gold modeling and retail KPI generation
04_Final_Validation_Testing	End-to-end validation and final test evidence

20.1 Notebook 01

The first notebook establishes the source boundary. It loads historical and incremental inputs, performs initial audit checks, and prepares Bronze representations.

20.2 Notebook 02

The second notebook contains the core transformation logic. It cleans records, standardizes fields, identifies changes, and uses MERGE patterns to update curated datasets.

20.3 Notebook 03

The third notebook creates the Gold layer, builds fact/dimension structures, and calculates the defined retail KPIs.

20.4 Notebook 04

The final notebook acts as a quality gate. It verifies table availability, counts, uniqueness, referential integrity, and KPI outputs.

21. Security, Governance and Reliability

Enterprise data platforms require governance in addition to transformation logic. The project uses the Databricks ecosystem and Unity Catalog-oriented organization to establish controlled data objects and improve discoverability.

21.1 Governance Principles

- Use clearly named catalogs, schemas, and tables.
- Separate raw and curated data responsibilities.
- Limit unnecessary access to sensitive datasets.
- Document business definitions for analytical outputs.
- Maintain reproducibility through version control.

21.2 Reliability Principles

- Use Delta transactions for controlled updates.
- Validate inputs before publishing downstream results.
- Keep transformation stages modular.
- Design for reruns and incremental delivery.
- Capture validation evidence during final testing.

21.3 Future Security Enhancements

A production deployment could extend this approach with role-based access control, environment separation, secret management, audit logging, row/column-level controls, and formal data-retention policies.

22. Testing Strategy

Testing is performed at multiple levels so that a failure can be localized to the appropriate stage.

Testing Level	Purpose	Example
Ingestion Test	Verify source load	File read and row availability
Schema Test	Verify structure	Expected columns and datatypes
Data Quality Test	Verify content	Nulls, duplicates, valid values
Transformation Test	Verify logic	Derived columns and joins
Merge Test	Verify incremental logic	Insert/update behavior
SCD Test	Verify history	Old and new customer versions
KPI Test	Verify business logic	Expected aggregation behavior
End-to-End Test	Verify pipeline	All layers complete successfully

22.1 Test Philosophy

A successful pipeline run is not sufficient evidence by itself. The final validation should demonstrate that the resulting data is structurally correct, logically consistent, and suitable for the intended analytical use.

23. Validation Results

The final validation stage is intended to provide evidence that the project outputs meet the defined requirements. The report should be accompanied by screenshots or exported evidence from the final Databricks notebook.

Validation Area	Expected Status	Evidence to Attach
Bronze tables available	PASS	Table/output screenshot
Silver transformations completed	PASS	Notebook output
MERGE execution	PASS	Execution/result output
SCD customer logic	PASS	Current/history record sample
Gold tables available	PASS	Catalog/table screenshot
Regional Net Margin	PASS	KPI query output
Promotional AOV	PASS	KPI query output
Customer Churn	PASS	KPI query output
Store Traffic	PASS	KPI query output
Final audit checks	PASS	Validation notebook output

23.1 Submission Evidence Checklist

- Insert final architecture screenshot.
- Insert Bronze table screenshot.
- Insert Silver transformation screenshot.
- Insert Gold star-schema/KPI screenshot.
- Insert final validation PASS screenshot.
- Verify all screenshots match the final GitHub version.

24. Performance Considerations

Although the project is an internship-scale implementation, its architecture reflects patterns that can scale to larger workloads.

24.1 Distributed Processing

Apache Spark distributes data processing across executors, making it suitable for large retail datasets. Transformations should be designed to minimize unnecessary shuffles and repeated scans.

24.2 Incremental Processing

MERGE-based incremental processing avoids rebuilding complete target tables when only a subset of records changes.

24.3 Partitioning and File Management

For larger production datasets, partitioning can be considered on frequently filtered columns such as transaction date or region, subject to workload characteristics. Poor partitioning can create small files or excessive partitions, so it should be driven by measured query patterns.

24.4 Query Optimization

- Select only required columns.
- Filter early where safe.
- Avoid repeated expensive transformations.
- Reuse curated datasets for multiple KPIs.
- Monitor data volume and execution time.

25. Challenges and Solutions

Challenge	Impact	Solution / Approach
Inconsistent source formats	Transformation errors	Explicit schema and datatype handling
Duplicate records	Incorrect aggregations	Business-key based deduplication
Null values	Broken joins or KPIs	Field-specific null rules
Changing customer attributes	Loss of history	SCD Type 2
Incremental updates	Duplicate/incorrect target state	Delta MERGE
Invalid records	Unreliable KPIs	Audit validation gate
Multiple KPI definitions	Inconsistent reports	Centralized Gold logic
Complexity across stages	Maintenance difficulty	Purpose-specific notebooks

25.1 Key Engineering Lesson

Most data engineering challenges are not solved by one transformation. Reliability comes from the combination of architecture, validation, data modeling, incremental processing, and clear business definitions.

26. Business Value

Apex Retail Intelligence creates value by turning operational retail records into reusable analytical information.

26.1 Management Value

- Regional profitability can be compared consistently.
- Promotion performance can be assessed using a common AOV definition.
- Customer inactivity can be identified for retention programs.
- Store traffic can be monitored alongside sales.

26.2 Data Team Value

- Centralized transformation logic.
- Reduced repeated data cleaning.
- Reusable Gold datasets.
- Traceable quality controls.
- Incremental processing patterns suitable for future automation.

26.3 BI Value

A BI team can connect reporting tools to curated Gold structures rather than building independent transformations in every dashboard. This improves consistency and makes future dashboard development faster.

27. Limitations and Future Scope

27.1 Current Limitations

- The core implementation is batch-oriented rather than real-time.
- Production orchestration and alerting can be expanded.
- Advanced anomaly detection is not part of the core solution.
- The project does not implement a complete enterprise BI dashboard.
- Security configuration should be strengthened for production deployment.

27.2 Future Enhancements

20. Add Azure Data Factory or Databricks Workflows for production orchestration.
21. Introduce Auto Loader for scalable file ingestion.
22. Add structured streaming for near-real-time retail events.
23. Create Power BI dashboards over Gold KPI tables.
24. Implement automated data-quality monitoring and alerts.
25. Add forecast models for sales, demand, and customer churn.
26. Introduce a feature store or ML pipeline for predictive analytics.
27. Add CI/CD for notebook and SQL deployment.
28. Introduce data lineage and automated documentation.
29. Expand the dimensional model with supplier, employee, campaign, and inventory dimensions.

28. Conclusion

Apex Retail Intelligence demonstrates how a retail data platform can be designed from ingestion to business insight using modern data engineering practices. The project moves beyond basic CSV processing by introducing a layered architecture, reliable Delta-based storage, transformation controls, incremental MERGE operations, dimensional modeling, SCD handling, audit validation, and reusable KPI datasets.

The four core KPIs—Regional Net Margin, Promotional Effectiveness, Customer Churn, and Store Traffic—illustrate the final purpose of the platform: enabling business questions to be answered using trusted and consistently prepared data.

The project also demonstrates the importance of separating raw preservation from transformation and analytical consumption. This separation makes the platform easier to debug, maintain, extend, and govern.

As a final internship project, Apex Retail Intelligence connects the technical concepts learned during the Celebal Excellence Internship into one integrated solution. It provides a strong foundation for future production enhancements such as orchestration, real-time processing, BI dashboards, advanced data quality monitoring, and predictive analytics.

29. Learning Outcomes

The project provided hands-on exposure to the following areas:

- Understanding of enterprise data-engineering workflows.
- Practical use of Databricks notebooks and compute.
- Apache Spark-based transformation concepts.
- Delta Lake tables and transactional updates.
- Medallion Architecture and data-layer responsibilities.
- Incremental loading and MERGE operations.
- Star-schema and dimensional modeling.
- Slowly Changing Dimensions, including Type 1 and Type 2.
- Data quality checks and validation gates.
- Business KPI design and analytical SQL.
- Documentation and repository organization.
- Thinking about scalability, reliability, governance, and maintainability.

29.1 Internship-to-Project Connection

The project connects the internship learning progression—from Python and SQL foundations through ETL/ELT, Azure concepts, Spark, Databricks, Delta Lake, and modeling—into a single business-oriented implementation.

30. References

- Databricks documentation and learning resources for notebooks, Spark, Delta Lake and data engineering.
- Apache Spark documentation for distributed data processing and SQL/DataFrame operations.
- Delta Lake documentation for transactions, MERGE and table management.
- General dimensional modeling concepts for fact and dimension design.
- Internship learning materials and assignments provided through the Celebal Technologies Excellence Internship Program.

30.1 Project Repository

Insert the final GitHub repository URL here before submission.

30.2 Final Submission Note

All screenshots, architecture figures, notebook names, KPI outputs, and validation evidence included in the final submission should correspond to the final committed version of the project.

Appendix A – Suggested Repository Structure

The following structure is recommended for the final project repository:

Path	Purpose
README.md	Project overview and execution guide
documentation/	Architecture and detailed documentation
documentation/Architecture.png	Architecture diagram
documentation/screenshots/	Execution and validation screenshots
notebooks/01_Raw_Landing_Audit	Raw/Bronze and audit workflow
notebooks/02_Silver_Transformation_MERGE	Silver and incremental processing
notebooks/03_Gold_StarSchema_KPIs	Gold and KPI generation
notebooks/04_Final_Validation_Testing	Final validation
data/	Optional sample/non-sensitive data
LICENSE	Repository licensing information

Appendix B – KPI Data Dictionary

KPI	Grain	Measure	Business Definition
Regional Net Margin	Region / period	Revenue - Cost	Profit contribution by region
Regional Net Margin %	Region / period	Net Margin / Revenue	Profitability ratio
Promoted AOV	Promotion status / period	Order Value / Orders	Average promoted basket
Non-Promoted AOV	Promotion status / period	Order Value / Orders	Baseline average basket
Customer Churn	Customer / period	Churn flag	Inactivity according to defined threshold
Store Traffic	Store / period	Traffic count	Total valid store activity

Appendix C – Data Quality Checklist

30. Confirm all expected source files are present.
31. Confirm required columns exist.
32. Confirm business keys are not null.
33. Check duplicates using appropriate business keys.
34. Validate numeric and date fields.
35. Check referential integrity between facts and dimensions.
36. Verify Silver transformations.
37. Verify MERGE insert/update behavior.
38. Verify customer SCD Type 2 history.
39. Verify product Type 1 updates.
40. Verify Gold table availability.
41. Verify KPI calculations.
42. Run final validation notebook.
43. Capture PASS/FAIL evidence.
44. Commit the final project version to GitHub.

Appendix D – Sample SQL / PySpark Logic Patterns

The following are conceptual patterns to illustrate the implementation approach. They should be replaced with the exact final project code snippets if the report requires source-code evidence.

D.1 Aggregation Pattern

Group transactional records by the required business dimension, calculate SUM/COUNT measures, and derive KPI ratios only after validating denominators.

D.2 MERGE Pattern

Match incoming records to target records using a stable business key; update changed records and insert new records. For SCD Type 2, close the current record before inserting the new version.

D.3 Validation Pattern

Calculate a test result, assign PASS or FAIL, collect results into an audit summary, and stop downstream execution when a critical test fails.

Appendix E – Presentation Summary

Apex Retail Intelligence is an end-to-end retail data engineering platform built using Databricks, Spark and Delta Lake. The project follows a Medallion Architecture in which source data moves from Raw/Landing to Bronze, Silver and Gold layers.

Bronze preserves source-aligned data. Silver performs cleansing, standardization, deduplication, incremental processing and business transformations. Delta MERGE is used for inserts and updates, while SCD Type 2 preserves customer history and Type 1 handles product updates.

The Gold layer uses dimensional modeling and produces business-ready datasets. Four key KPIs are generated: Regional Net Margin, Promotional Effectiveness through AOV, Customer Churn, and Store Traffic.

The project also includes an audit validation framework that produces PASS/FAIL outcomes and protects downstream analytics from critical data-quality failures.

One-Minute Explanation

“My final project is Apex Retail Intelligence, an enterprise-style retail data engineering platform built on Databricks. I designed it using Medallion Architecture with Raw, Bronze, Silver and Gold layers. The raw data is ingested into Bronze, cleaned and transformed in Silver, and then modeled into a Gold star schema. I implemented incremental loading using Delta MERGE and used SCD Type 2 for customer history and Type 1 for product updates. Finally, I created four retail KPIs—Regional Net Margin, Promotional Effectiveness, Customer Churn and Store Traffic—and added an audit validation layer with PASS/FAIL checks. The main objective was to create a reliable, reusable and BI-ready retail data platform rather than just a collection of SQL queries.”

Appendix F – Final Review Checklist

- Project title is consistently written as Apex Retail Intelligence.
- Internship name is consistently written as Celebal Technologies Excellence Internship Program.
- Final architecture image is inserted.
- Final notebook names match the GitHub repository.
- KPI definitions match the implemented SQL.
- PASS/FAIL validation evidence is inserted.
- GitHub repository URL is added.
- Screenshots are clear and readable.
- No confidential credentials or secrets are included.
- README instructions are complete.
- Final report and repository are synchronized.

Project Notes

```
04_Final_Validation_Testing x +
File Edit View Run Help Python ☆ Last edit was 1 hour ago Run all Serverless Schedule Comments Share

11:50 AM (<1s) 37: 9.3: Project Completion Summary
29 print(f"✅ SCD Type 2 implemented for Customer dimension")
30 print(f"✅ SCD Type 1 implemented for Product dimension")
31 print(f"✅ Star Schema with surrogate keys created")
32 print(f"✅ Unity Catalog integration completed")
33 print(f"✅ All 5 required KPIs delivered")
34 print(f"✅ Comprehensive testing and validation performed")
35 print(f"✅ Full documentation and evidence captured")
36
37 print("\n" + "="*80)
38 print(" PHASE 4 VALIDATION COMPLETE!")
39 print("="*80)

=====
PROJECT COMPLETION SUMMARY
=====

Timeline:
Test Started: 2026-08-09 06:20:43
Test Completed: 2026-08-09 06:21:36
Duration: 0:00:52.496104

Project Structure:
Notebooks Created: 4
Layers Implemented: 5 (Raw, Landing, Bronze, Silver, Gold)
Delta Tables Created: 11
Unity Catalog Tables: 5
KPIs Generated: 5

Data Volume:
bronze_customers: 2,105 records
bronze_products: 2,084 records
bronze_sales: 2,002 records
```

```
04_Final_Validation_Testing x +
File Edit View Run Help Python ☆ Last edit was 1 hour ago Run all Serverless Schedule Comments Share

11:50 AM (1s) 35: 9.1: Final Compliance Audit Python
79 overall_pct = (overall_complete / overall_total * 100) if overall_total > 0 else 0
80
81 print("="*80)
82 print(f"OVERALL COMPLETION: {overall_complete}/{overall_total} ({overall_pct:.0f}%)")
83 print("="*80)
84
85 assert_test(
86     overall_pct == 100,
87     "Overall Assignment Compliance",
88     f"All {overall_total} requirements met ({overall_pct:.0f}%)",
89     f"Only {overall_complete}/{overall_total} requirements met"
90 )

=====
FINAL COMPLIANCE AUDIT
=====

COMPLIANCE STATUS BY PHASE:
=====

Phase 1:
✅ Raw data loaded into Landing layer
✅ Audit log implemented and captures all ingestion
✅ File-level audit trail maintained

Phase 2:
✅ Bronze layer created with all raw data
✅ Silver layer implements SCD Type 2 for Customer
✅ Silver layer implements SCD Type 1 for Product
✅ Sales data immutable with surrogate keys
✅ Delta Lake format used throughout
```

```
11:50 AM (<1s) 36: 9.2: Test Results Summary

1 # Summarize all test results
2 print("\n" + "="*80)
3 print("📊 TEST RESULTS SUMMARY")
4 print("="*80)
5
6 if len(test_results) > 0:
7     passed_tests = [r for r in test_results if r["status"] == "PASS"]
8     failed_tests = [r for r in test_results if r["status"] == "FAIL"]
9
10    print(f"\nTotal Tests Run: {len(test_results)}")
11    print(f"Passed: {len(passed_tests)} ✅")
12    print(f"Failed: {len(failed_tests)} ❌")
13
14    success_rate = (len(passed_tests) / len(test_results) * 100) if len(test_results) > 0 else 0
15    print(f"Success Rate: {success_rate:.1f}%")
16
17    if failed_tests:
18        print("\n❌ FAILED TESTS:")
19        print("="*80)
20        for test in failed_tests:
21            print(f"\nTest: {test['test_name']}")
22            print(f"Message: {test['message']}")
23            if test['details']:
24                print(f"Details: {test['details']}")
25    else:
26        print("\n🎉 ALL TESTS PASSED! 🎉")
27 else:
28    print("\nNo tests recorded. Please run validation sections.")
29
30 print("\n" + "="*80)
```

```
=====
📊 TEST RESULTS SUMMARY
=====

Total Tests Run: 44
Passed: 44 ✅
Failed: 0 ❌
Success Rate: 100.0%

🎉 ALL TESTS PASSED! 🎉
=====
```

```
gold_dim_customer: 2,104 records
gold_dim_product: 2,083 records
gold_dim_promotion: 1,570 records
gold_dim_date: 2,000 records
gold_fact_sales: 2,000 records

✅ Assignment Status: COMPLETE

Key Achievements:
✅ Medallion Architecture (Bronze-Silver-Gold) implemented
✅ SCD Type 2 implemented for Customer dimension
✅ SCD Type 1 implemented for Product dimension
✅ Star Schema with surrogate keys created
✅ Unity Catalog integration completed
✅ All 5 required KPIs delivered
✅ Comprehensive testing and validation performed
✅ Full documentation and evidence captured

=====
PHASE 4 VALIDATION COMPLETE!
=====
```

```
11:50 AM (<1s) 37: 9.3: Project Completion Summary Python

1 # Project completion summary
2 test_end_time = datetime.now()
3 test_duration = test_end_time - test_start_time
4
5 print("\n" + "="*80)
6 print(" PROJECT COMPLETION SUMMARY")
7 print("="*80)
8
9 print(f"\n Timeline:")
10 print(f" Test Started: {test_start_time.strftime('%Y-%m-%d %H:%M:%S')}")
11 print(f" Test Completed: {test_end_time.strftime('%Y-%m-%d %H:%M:%S')}")
12 print(f" Duration: {test_duration}")
13
14 print(f"\n Project Structure:")
15 print(f" Notebooks Created: 4")
16 print(f" Layers Implemented: 5 (Raw, Landing, Bronze, Silver, Gold)")
17 print(f" Delta Tables Created: 11")
18 print(f" Unity Catalog Tables: 5")
19 print(f" KPIs Generated: 5")
20
21 print(f"\n Data Volume:")
22 for metric, value in baseline_metrics.items():
23     if value > 0:
24         print(f" {metric}: {value:,} records")
25
26 print(f"\n✅ Assignment Status: COMPLETE")
27 print(f"\n Key Achievements:")
28 print(f" ✅ Medallion Architecture (Bronze-Silver-Gold) implemented")
29 print(f" ✅ SCD Type 2 implemented for Customer dimension")
30 print(f" ✅ SCD Type 1 implemented for Product dimension")
31 print(f" ✅ Star Schema with surrogate keys created")
```